

CSC 4520/6520

Design & Analysis of Algorithms

CHAPTER 7: SPACE AND TIME TRADE-OFFS

Abdullah Bal, PhD

Objectives



Recap

Heaps and Heapsort

Problem Reduction

LCM

Optimization



Space-Time Trade-offs



Counting Sort Horspool String Matching



Hashing

Space and Time Trade-Offs

- Two varieties of space-for-time algorithms:

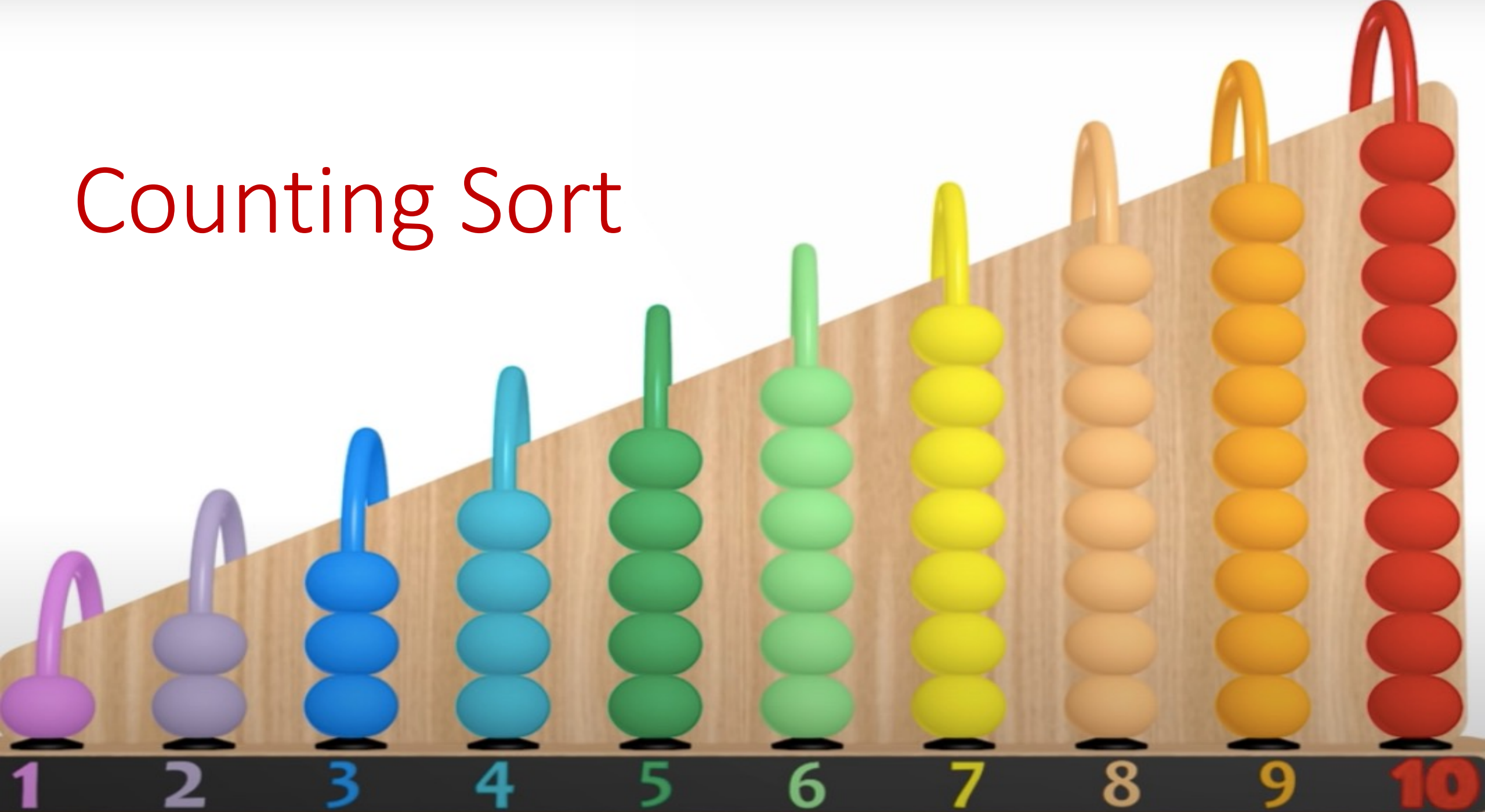
Input enhancement — preprocess the input (or its part) to store some info to be used later in solving the problem

- Counting sorts
- String searching algorithms

Prestructuring — preprocess the input to make accessing its elements easier

- hashing
- indexing schemes (e.g., B-trees)

Counting Sort



Sorting by Counting

- Counting sort is an example of **input enhancement** algorithm design techniques.
- Assume, for example, that we have to sort a list whose values can be either 1 or 2.
- We can scan the list to **compute the number of 1's** and **the number of 2's in it** and then, on the second pass, simply make the appropriate number of the first elements equal to 1 and the remaining elements equal to 2.
- More generally, if element values are integers between some lower bound l and upper bound u , we can **compute the frequency** of each of those values and **store them** in array $F[0..u - l]$.
- Then the first $F[0]$ positions in the sorted list must be filled with l , the next $F[1]$ positions with $l + 1$, and so on.
- All this can be done, of course, only if we can overwrite the given elements.

Sorting by Counting

- Let us consider a more realistic situation of sorting a list of items with some other information associated with their keys so that we cannot overwrite the list's elements.
- Then we can copy elements into a new array $S[0..n-1]$ to hold the sorted list as follows.
- The elements of A whose values are equal to the lowest possible value l are copied into the first $F[l]$ elements of S , i.e., positions 0 through $F[l]-1$; the elements of value $l + 1$ are copied to positions from $F[l]$ to $(F[l] + F[l+1]) - 1$; and so on.
- Since such accumulated sums of frequencies are called a distribution in statistics, the method itself is known as *distribution counting*.

Example 1: Sorting by Counting (First Alg.)

- Consider sorting the array

13	11	12	13	12	12
----	----	----	----	----	----

whose values are known to come from the set {11, 12, 13} and should not be overwritten in the process of sorting.

- The frequency and distribution arrays are as follows:

Array values	11	12	13
Frequencies	1	3	2
Distribution values	1	4	6

- Note that the distribution values indicate the proper positions for the last occurrences of their elements in the final sorted array.
- If we index array positions from 0 to $n - 1$, the distribution values must be reduced by 1 to get corresponding element positions.

Example 1: Sorting by Counting (First Alg.)

- It is more convenient to process the input array right to left.

13	11	12	13	12	12
----	----	----	----	----	----
- For the example, the last element is 12, and, since its distribution value is 4, we place this 12 in position $4-1 = 3$ of the array S that will hold the sorted list.
- Then we decrease the 12's distribution value by 1 and proceed to the next (from the right) element in the given array.

Iteration	Sort List					
1				12		
2			12	12		
3			12	12		13
4		12	12	12		13
5	11	12	12	12		13
6	11	12	12	12	13	13

Sorting by Counting

ALGORITHM *DistributionCountingSort*($A[0..n-1]$, l , u)

//Sorts an array of integers from a limited range by distribution counting

//Input: An array $A[0..n-1]$ of integers between l and u ($l \leq u$)

//Output: Array $S[0..n-1]$ of A 's elements sorted in nondecreasing order

for $j \leftarrow 0$ **to** $u - l$ **do** $D[j] \leftarrow 0$ //initialize frequencies

for $i \leftarrow 0$ **to** $n - 1$ **do** $D[A[i] - l] \leftarrow D[A[i] - l] + 1$ //compute frequencies

for $j \leftarrow 1$ **to** $u - l$ **do** $D[j] \leftarrow D[j - 1] + D[j]$ //reuse for distribution

for $i \leftarrow n - 1$ **downto** 0 **do**

$j \leftarrow A[i] - l$

$S[D[j] - 1] \leftarrow A[i]$

$D[j] \leftarrow D[j] - 1$

return S

Example 1: Sorting by Counting (Second Alg.)

- Consider sorting the array

13	11	12	13	12	12
----	----	----	----	----	----

whose values are known to come from the set {11, 12, 13} and should not be overwritten in the process of sorting.

- The frequency array is as follows:

Array values	11	12	13
Frequencies	1	3	2

- The sorted array according to frequency list as follows:

11	12	12	12	13	13
----	----	----	----	----	----

Example 2: Sorting by Counting

Sort the following array using counting sort:

1	0	2	5	3	7	3	4	0	6
---	---	---	---	---	---	---	---	---	---

Analysis of Counting Sort

- Assuming that the range of array values is fixed, this is obviously a **linear algorithm** because it makes just **two consecutive passes** through its input array A $\Theta(n+k)$
- This is a **better time-efficiency class** than that of the most efficient sorting algorithms—**mergesort, quicksort, and heapsort**—we have encountered.
- It is important to remember, however, that this efficiency is obtained by exploiting the specific nature of inputs for which sorting by distribution counting works, in addition **to trading space for time**.



String Searching

Review: String searching by brute force

pattern: a string of m characters to search for

text: a (long) string of n characters to search in

Brute force algorithm

Step 1 Align pattern at beginning of text

Step 2 Moving from left to right, compare each character of pattern to the corresponding character in text until either all characters are found to match (successful search) or a mismatch is detected

Step 3 While a mismatch is detected and the text is not yet exhausted, realign pattern one position to the right and repeat Step 2

The time complexity of brute force is $O(n*m)$

String searching by preprocessing

- There are several string searching algorithms based on the input enhancement idea of preprocessing the pattern
 - Boyer - Moore algorithm preprocesses pattern right to left and **store information into two tables**
 - Horspool's algorithm simplifies the Boyer-Moore algorithm by using just **one table**
 - Knuth-Morris-Pratt (KMP) algorithm preprocesses pattern left to right to get useful information for later searching

Horspool's Algorithm

- Preprocesses pattern to **generate a shift table** that determines how much to shift the pattern when a **mismatch occurs**.
- Always makes a shift based on **the text's character c** aligned with the last character in the pattern according to the shift table's entry for c .

Horspool's Algorithm

- If a mismatch occurs, we need to shift the pattern to the right.
- Clearly, we would like to make as large a shift as possible without risking the possibility of missing a matching substring in the text.
- Horspool's algorithm determines the size of such a shift by looking at the character c of the text that is aligned against the last character of the pattern.
- This is the case even if character c itself matches its counterpart in the pattern.

Horspool's Algorithm

- In general, the following **four possibilities** can occur.

Case 1:

- If there are no c's in the pattern—e.g., c is letter S in our example—we can safely shift the pattern by its entire length

s_0 ... S ... s_{n-1}

 X

 B A R B E R

 B A R B E R

Horspool's Algorithm

Case2:

- If there are occurrences of character c in the pattern but it is not the last one there—e.g., c is letter B in our example—the shift should align the rightmost occurrence of c in the pattern with the c in the text:

$$\begin{array}{ccccccccccc}
 s_0 & & \dots & & & & B & & \dots & & s_{n-1} \\
 & & & & & & \diagdown & & & & \\
 & & & & & & & & & & \\
 & & & & & & B & A & R & B & E & R \\
 & & & & & & & & & & & \\
 & & & & & & & & & & B & A & R & B & E & R
 \end{array}$$

Horspool's Algorithm

Case3 :

- If c happens to be the last character in the pattern but there are no c 's among its other $m - 1$ characters—e.g., c is letter R in our example—the situation is similar to that of **Case 1** and the pattern should be shifted by the entire pattern's length m :

s_0 ... M E R ... s_{n-1}

 X || ||

 L E A D E R

 L E A D E R

Horspool's Algorithm

Case 4 :

- If c happens to be the last character in the pattern and there are other c 's among its first $m - 1$ characters—e.g., c is letter R in our example— the situation is similar to that of **Case 2** and the **rightmost occurrence of c** among the first $m - 1$ characters in the pattern should be aligned with the text's c :

$$\begin{array}{cccccccccccccccc}
s_0 & & \dots & & & & & A & R & & & \dots & & s_{n-1} \\
& & & & & & & \parallel & \parallel & & & & & \\
& & & & & & & R & E & O & R & D & E & R \\
& & & & & & & & & & R & E & O & R & D & E & R
\end{array}$$

Horspool's Algorithm

- These examples clearly demonstrate that right-to-left character comparisons can lead to **farther shifts of the pattern than the shifts by only one position** always made by the brute-force algorithm.
- However, if such an algorithm had to check all the characters of the pattern on every trial, it would lose much of this superiority.
- Fortunately, the idea of input enhancement makes **repetitive comparisons unnecessary**.
- We can **precompute** shift sizes and store them in a table.

Shift Table

- The table's entries will indicate the shift sizes computed by the formula

$$t(c) = \begin{cases} \text{The pattern's length } m, \text{ if } c \text{ is not among the first } m - 1 \text{ characters of the} \\ \text{pattern which are represented with a character } **; \\ \\ \text{The distance from the rightmost } c \text{ among the first } m - 1 \text{ characters of the} \\ \text{pattern to its last character, otherwise.} \end{cases}$$

Shift Table

- For example, for the pattern BARBER, “**” will be equal to 6, except for the entries for E, B, R, and A, which will be 1, 2, 3, and 4, respectively.

Shift Table

Character c	B	A	R	E	**
Shift t(c)	2	4	3	1	6

- Scan the pattern right to left starting second endmost character. Compute the distance from c to end of the pattern.

Horspool's Algorithm

Step 1 : For a given pattern of length m and the alphabet used in both the pattern and text, construct the shift table as described above.

Step 2 : Align the pattern against the beginning of the text.

Step 3 : Repeat the following until either a matching substring is found, or the pattern reaches beyond the last character of the text.

- Starting with the last character in the pattern, compare the corresponding characters in the pattern and text until either all m characters are matched (then stop) or a mismatching pair is encountered.
- In the latter case, retrieve the entry $t(c)$ from the c 's column of the shift table where c is the text's character currently aligned against the last character of the pattern, and shift the pattern by $t(c)$ characters to the right along the text.

Example 1: String Matching

- Consider searching for the pattern BARBER in a text that comprises English letters and spaces (denoted by underscores).
- The shift table, as we mentioned, is filled as follows:

Character c	B	A	R	E	**
Shift t(c)	2	4	3	1	6

```
J I M _ S A W _ M E _ I N _ A _ B A R B E R S H O P
B A R B E R           B A R B E R
      B A R B E R       B A R B E R
        B A R B E R         B A R B E R
```

Example 2: String Matching

Text: DESIGN & ANALYSIS OF ALGORITHMS

Pattern: ALGO

Example 2: String Matching

Text: DESIGN & ANALYSIS OF ALGORITHMS

Pattern: ALGO

Character c	A	L	G	**
Shift t(c)	3	2	1	4

D	E	S	I	G	N		&		A	N	A	L	Y	S	I	S		O	F		A	L	G	O	R	I	T	H	M	S
A	L	G	O																											
				A	L	G	O																							
								A	L	G	O																			
											A	L	G	O																
														A	L	G	O													
																	A	L	G	O										
																		A	L	G	O									
																				A	L	G	O							

Example 3: String Matching

Text: GEORGIA STATE UNIVERSITY COMPUTER SCIENCE

Pattern: SCIENCE

Example 3: String Matching

Text: GEORGIA STATE UNIVERSITY COMPUTER SCIENCE

Pattern: SCIENCE

Character c	S	C	I	E	N	**
Shift t(c)	6	1	4	3	2	7

G	E	O	R	G	I	A		S	T	A	T	E		U	N	I	V	E	R	S	I	T	Y		C	O	M	P	U	T	E	R		S	C	I	E	N	C	E			
S	C	I	E	N	C	E																																					
							S	C	I	E	N	C	E																														
														S	C	I	E	N	C	E																							
																				S	C	I	E	N	C	E																	
																												S	C	I	E	N	C	E									
																																				S	C	I	E	N	C	E	

Example 4: String Matching

Text: WELCOME TO ART COLLEGE

Pattern: COLL

Example 4: String Matching

Text: WELCOME TO ART COLLEGE

Pattern: COLL

Character c				C	O				L				**								
Shift t(c)				3	2				1				4								
W	E	L	C	O	M	E		T	O		A	R	T		C	O	L	L	E	G	E
C	O	L	L																		
			C	O	L	L															
							C	O	L	L											
											C	O	L	L							
															C	O	L	L			

Analysis of Horspool's Algorithm

- A simple example can demonstrate that the worst-case efficiency of Horspool's algorithm is in $O(nm)$.
- But for random texts, it is in $\Theta(n)$, and, although in the same efficiency class, Horspool's algorithm is obviously faster on average than the brute-force algorithm.
- In fact, as mentioned, it is often at least as efficient as its more sophisticated predecessor discovered by R. Boyer and J. Moore.

Boyer-Moore algorithm

- Based on same two ideas:
 - Comparing pattern characters to text from right to left
 - Precomputing shift sizes in **two tables**
 - **Bad-symbol table** indicates how much to shift based on text's character causing a mismatch
 - **Good-suffix table** indicates how much to shift based on matched part (suffix) of the pattern

Boyer-Moore Algorithm Tables

Good-suffix Table

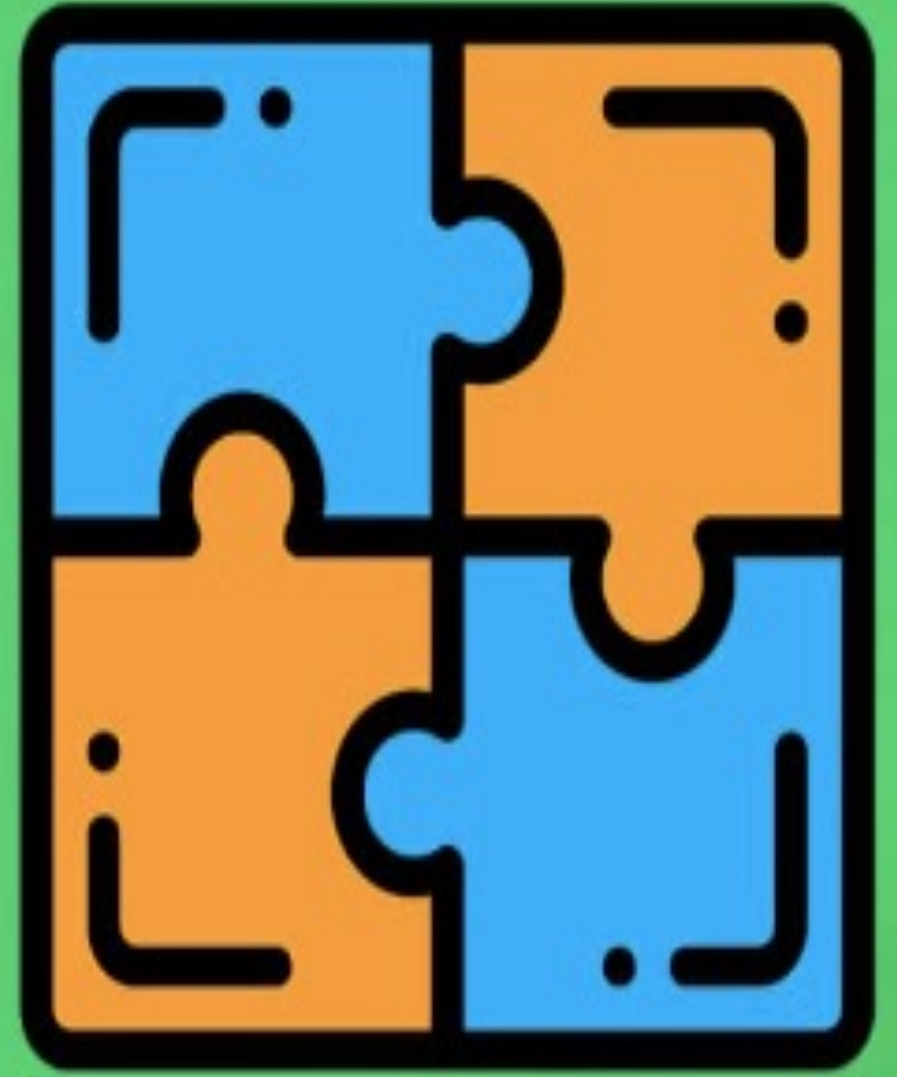
k	pattern	d_2
1	BAO B AB	2
2	B AOB A B	5
3	B AO B AB	5
4	B A O BAB	5
5	B A O BAB	5

Bad-symbol Table

Character c	B	A	O	**
Shift t(c)	2	1	3	6



Hashing



Hashing

- The idea of hashing is to map keys of a given file of size n into a table of size m , called the hash table, by using a predefined function, called the hash function.
- It's discovered in the 1950s by IBM researchers.
- A very efficient method for implementing a *dictionary*, (student records in a school, citizen records in a governmental office, book records in a library)

Hashing

- A set with the operations:
 - find
 - insert
 - delete
- Based on representation-change and space-for-time tradeoff ideas
- Important applications:
 - symbol tables
 - databases (*extendible hashing*)

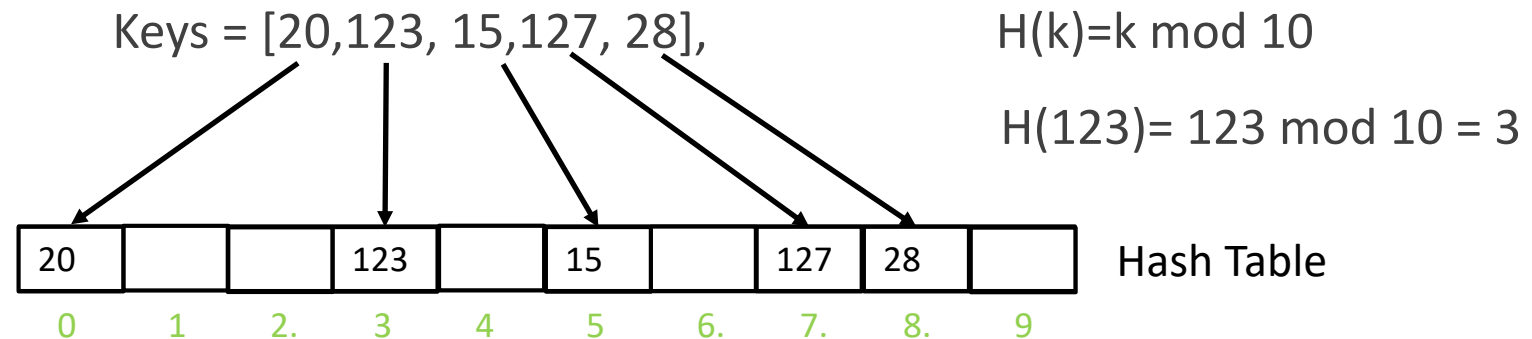
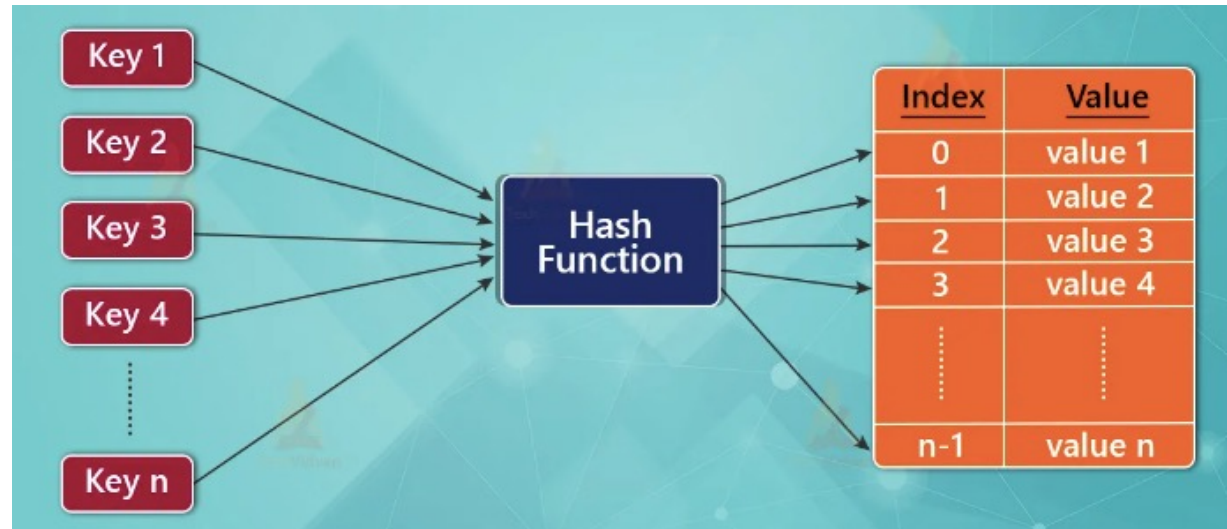
Hash tables and hash functions

- Typically, records comprise several fields, each responsible for keeping a particular type of information about an entity the record represents.
- For example, a student record may contain fields for the student's ID, name, date of birth, home address, major, and so on.
- Among record fields there is usually at least one called a **key** that is used for identifying entities represented by the records (e.g., the student's ID).

Hash tables and hash functions

- We can assume that we have to implement a **dictionary** of n records with keys $K_1, K_2, \dots, K_n$.
- Hashing is based on the idea of **distributing keys** among a one-dimensional array $H[0..m - 1]$ called a ***hash table***.
- The **distribution** is done by computing, for each of the keys, the value of some predefined function h called the ***hash function***.
- This function assigns an integer between 0 and $m - 1$, called the ***hash address***, to a key.

Hash tables and hash functions



Hash functions

- A hash function can be a form of key

$$h(K) = K \bmod m$$

- If keys are **letters** of some alphabet, we can first assign a letter its position in the alphabet, denoted **ord(K)**, and then apply the same kind of a function used for integers.

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26

- Finally, if K is a character string $c_0 c_1 \dots c_{s-1}$, we can use, as a very unsophisticated option,

$$\sum_{i=0}^{s-1} \text{ord}(c_i) \bmod m$$

$$h(\text{ARE}) = (1 + 18 + 5) \bmod 13 = 11$$

$$h(\text{SOON}) = (19 + 15 + 15 + 14) \bmod 13 = 11.$$

Hash functions

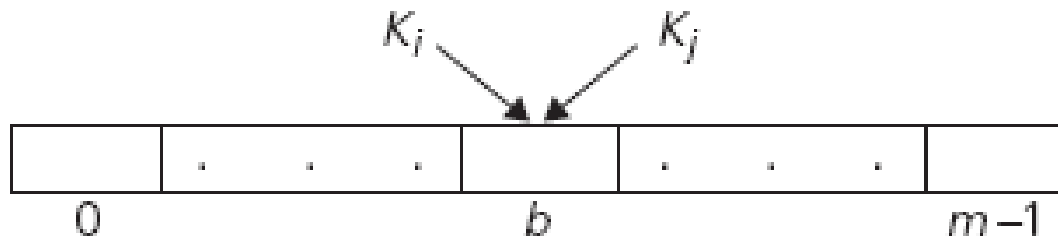
- In general, a hash function needs to satisfy somewhat conflicting requirements:
 - A hash table's size should **not be excessively large** compared to the **number of keys**, but it should be sufficient to not jeopardize the implementation's time efficiency.
 - A hash function needs to **distribute keys** among the cells of the hash table **as evenly as possible**.
 - A hash function has to be **easy to compute**.
- Example: student records, key = SSN.
- Hash function:

$h(K) = K \bmod m$ where m is some integer (typically, prime)

If $m = 1000$, where is record with SSN= 314159265 stored?

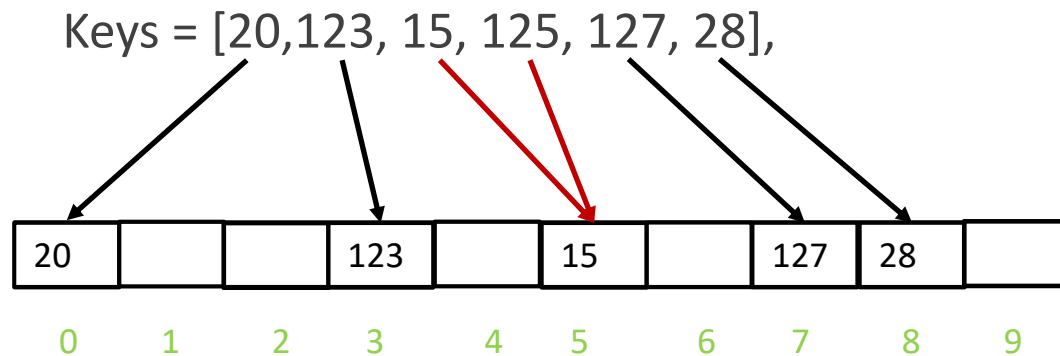
Collisions

- If we choose a hash table's size m to be smaller than the number of keys n , we will get **collisions**—a phenomenon of two (or more) keys being hashed into the same cell of the hash table: $h(K_i) = h(K_j)$.



- However, collisions should be expected even if m is considerably larger than n .
- In fact, in **the worst case**, all the keys could be hashed to the same cell of the hash table.
- Fortunately, with an appropriately chosen hash table size and a good hash function, this situation **happens very rarely**.

Collisions



$$H(k) = k \bmod 10$$

$$H(125) = 125 \bmod 10 = 5$$

Hash Table

Collisions

- Every hashing scheme must have a **collision resolution** mechanism.
- This mechanism is different in the two principal versions of hashing:
 - ***open hashing*** (also called ***separate chaining***)
 - each cell is a header of linked list of all keys hashed to it
 - ***closed hashing*** (also called ***open addressing***).
 - one key per cell
 - in case of collision, finds another cell by
 - *linear probing*: use next free bucket
 - *double hashing*: use second hash function to compute increment

Open hashing (Separate chaining)

- Keys are stored in **linked lists** attached to cells of a hash table.
- Each list contains all the keys hashed to its cell.
- Consider, as an example, the following list of words:

A, FOOL, AND, HIS, MONEY, ARE, SOON, PARTED

$h(K)$ = sum of K 's letters' positions in the alphabet MOD 13

Open hashing (Separate chaining)

A, FOOL, AND, HIS, MONEY, ARE, SOON, PARTED

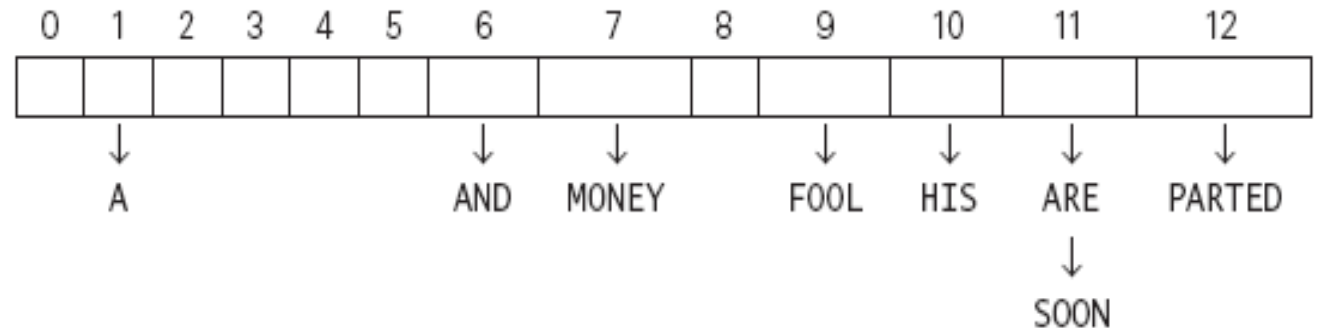
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26

- The first key is the word A; its hash value is $h(A) = 1 \bmod 13 = 1$.
- The second key—the word FOOL—is installed in the ninth cell since $(6 + 15 + 15 + 12) \bmod 13 = 9$
- There is a collision of the keys ARE and SOON because

$$h(ARE) = (1 + 18 + 5) \bmod 13 = 11$$

$$h(SOON) = (19 + 15 + 15 + 14) \bmod 13 = 11.$$

keys	A	FOOL	AND	HIS	MONEY	ARE	SOON	PARTED
hash addresses	1	9	6	10	7	11	11	12



Searching- Open hashing

- How do we search in a dictionary implemented as such a table of linked lists?
- We do this by simply applying to a search key the same procedure that was used for creating the table.
- To illustrate, if we want to search for the **key KID** in the hash table, we first compute the value of the same hash function for the key: $h(KID) = 11$.
- Since the list attached to cell 11 is not empty, its linked list may contain the search key.
- But because of possible collisions, we cannot tell whether this is the case until we traverse this linked list.
- After comparing the string KID first with the string ARE and then with the string SOON, we end up with an **unsuccessful search**.

Searching - Open hashing

- In general, the efficiency of searching depends on the **lengths of the linked lists**, which, in turn, depend on the dictionary and table sizes, as well as the **quality of the hash function**.
- If the hash function distributes **n keys among m cells** of the hash table about evenly, each list will be about n/m keys long.
- The ratio $\alpha = n/m$, called the ***load factor*** of the hash table, plays a crucial role in the efficiency of hashing.
- They are almost identical to **searching sequentially in a linked list**; what we have gained by hashing is a **reduction in average list size by a factor of m** , the size of the hash table.

Searching - Open hashing

- Normally, we want the load factor to be **not far from 1**.
- Having it too small would imply a lot of empty lists and hence inefficient use of space; having it too large would mean longer linked lists and hence longer search times.
- But if we do have the load factor around 1, we have an amazingly efficient scheme that makes it possible to search for a given key for, on average, the **price of one or two comparisons**!
- True, in addition to comparisons, we need to spend time on computing the value of the hash function for a search key, but it is a constant-time operation, independent from n and m .
- Note that we are getting this remarkable efficiency not only as a result of the method's ingenuity but also at the expense of extra space.

Insertion and Deletion - Open hashing

- The two other dictionary operations—insertion and deletion—are almost identical to searching.
- Insertions are normally done at the end of a list.
- Deletion is performed by searching for a key to be deleted and then removing it from its list.
- Hence, the efficiency of these operations is identical to that of searching, and they are all $\Theta(1)$ in the average case if the number of keys n is about equal to the hash table's size m .

Closed hashing

- In closed hashing, all keys are stored in the hash table itself without the use of linked lists.
- Different strategies can be employed for collision resolution.
 - **Linear probing:** use next free bucket
 - **Double hashing:** use second hash function to compute increment

Closed hashing – Linear Probing

- The simplest one—called ***linear probing***—checks the cell following the one where the collision occurs.
- If that cell is empty, the new key is installed there;
- If the next cell is already occupied, the availability of that cell's immediate successor is checked, and so on.
- If the end of the hash table is reached, the search is wrapped to the beginning of the table; i.e., it is treated as a circular array.

Closed hashing – Linear Probing

Linear probing: Keys are stored inside a hash table.

keys	A	FOOL	AND	HIS	MONEY	ARE	SOON	PARTED
hash addresses	1	9	6	10	7	11	11	12

0	1	2	3	4	5	6	7	8	9	10	11	12
	A											
	A								FOOL			
	A					AND			FOOL			
	A					AND			FOOL	HIS		
	A					AND	MONEY		FOOL	HIS		
	A					AND	MONEY		FOOL	HIS	ARE	
	A					AND	MONEY		FOOL	HIS	ARE	SOON
PARTED	A					AND	MONEY		FOOL	HIS	ARE	SOON
0	1	2	3	4	5	6	7	8	9	10	11	12
PARTED	A					AND	MONEY		FOOL	HIS	ARE	SOON

Searching - Linear Probing

- To search for a given key K , we start by computing $h(K)$ where h is the hash function used in the table construction.
- If the cell $h(K)$ is **empty**, the search is **unsuccessful**.
- If the cell is not empty, we must compare K with the **cell's occupant**: if they are equal, we have found a matching key;
- If they are not, we compare K with a key in the next cell and continue in this manner until we encounter either a matching key (a successful search) or an empty cell (unsuccessful search).

Searching - Linear Probing

- For example, if we search for the word LIT in the table, we will get

$$h(\text{LIT}) = (12 + 9 + 20) \bmod 13 = 2$$

- Since cell 2 is empty, we can stop immediately.

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26

0	1	2	3	4	5	6	7	8	9	10	11	12
PARTED	A					AND	MONEY		FOOL	HIS	ARE	SOON

Searching - Linear Probing

- If we search for KID with

$$h(KID) = (11 + 9 + 4) \bmod 13 = 11,$$

- We will have to compare KID with ARE, SOON, PARTED, and A before we can declare the search unsuccessful.

0	1	2	3	4	5	6	7	8	9	10	11	12
PARTED	A					AND	MONEY		FOOL	HIS	ARE	SOON

Deletion - Linear Probing

- Although the search and insertion operations are straightforward for this version of hashing, **deletion is not**.
- For example, if we **simply delete the key ARE** from the last state of the hash table, we will be **unable to find the key SOON** afterward.
- Indeed, after computing $h(\text{SOON}) = 11$, the algorithm would find this location empty and report the unsuccessful search result.
- A simple solution is to use “**lazy deletion**,” i.e., to mark previously occupied locations by a special symbol to **distinguish them from locations that have not been occupied**.

Linear Probing - Cluster Problem

- As the hash table gets closer to being full, the performance of linear probing deteriorates because of a phenomenon called clustering.
- A **cluster** in linear probing is a sequence of contiguously occupied cells.
- Clusters are bad news in hashing because they make the dictionary operations less efficient.
- As clusters become larger, the probability that a new element will be attached to a cluster increases;
- In addition, large clusters increase the probability that two clusters will coalesce after a new key's insertion, causing even more clustering.

Closed hashing – Double Hashing

- One of the collision resolution strategies is ***double hashing***.
- Under this scheme, we use another hash function, $s(K)$, to determine a fixed increment for the probing sequence to be used after a collision at location

$$l = h(K) \quad (l + s(K)) \bmod m, \quad (l + 2s(K)) \bmod m, \quad \dots\dots$$

- To guarantee that every location in the table is probed by this sequence, the increment $s(k)$ and the table size m must be relatively prime, i.e., their only common divisor must be 1.
- This condition is satisfied automatically if m itself is prime.
- Some functions recommended in the literature are

$$s(k) = m - 2 - k \bmod (m - 2) \text{ and } s(k) = 8 - (k \bmod 8) \text{ for small tables}$$

$$s(k) = k \bmod 97 + 1 \text{ for larger ones.}$$

Analysis of Closed hashing – Rehashing

- Mathematical analysis of double hashing has proved to be quite difficult.
- Some partial results and considerable practical experience with the method suggest that with good hashing functions—both primary and secondary—**double hashing is superior to linear probing.**
- But its performance also **deteriorates when the table gets close to being full.**
- A natural solution in such a situation is ***rehashing***: the current table is scanned, and all its keys are **relocated into a larger table.**

Analysis of Hashing

- It is worthwhile to compare the main properties of hashing **with balanced search trees**—its principal competitor for implementing dictionaries.
- **Asymptotic time efficiency** with hashing, searching, insertion, and deletion can be implemented to take $\Theta(1)$ time on the average but $\Theta(n)$ time in the very unlikely worst case.
- For balanced search trees, the average time efficiencies are $\Theta(\log n)$ for both the average and worst cases.

Summary

- Space-Time Trade-offs
- Counting Sort
- Horspool String Matching
- Hashing
- Open Hashing
- Closed Hashing

Supportive Materials

- *Introduction to The Design and Analysis of Algorithms (3rd Edition)* by Anany Levitin, Pearson.
- *Introduction to Algorithms*, by T.Cormen, C. Leiserson, R.Rivest and C.Stein, MIT Press, 3rd Edition.